MODELING HUMAN ACTIVITY STATES USING HIDDEN MARKOV MODELS

Prepared by	Carla BATONI
Coursework	Formative 2 - Hidden Markov Models
Github repository	https://github.com/Batonicarla/HMM-

1. Background and motivation

Human Activity Recognition (HAR) is the task of recognizing activities from the measurements taken by the sensor logger. It has applications in smart homes, smart environments, fitness tracking, and fall prevention. In this project, I explored the use of accelerometer and gyroscope data from a smartphone to classify the following activities: standing, walking, jumping, and still. Using the patterns of sensor-derived features, I created a Hidden Markov Model (HMM) in Python to better understand how the sensor data changes with each of the activities and to infer the most probable sequence of activities. The goal was to determine if the model could accurately classify activities in unseen recordings, and to gain insight into the difficulties encountered due to similarity of movements, sensor noise, and small training sets.

2. Data collection and preprocessing

It loads ZIP files that contain Accelerometer.csv and Gyroscope.csv into my notebook. The files are labelled with the activity they represent, and any files that start with “test” are separated for evaluation. There were 62 zip files found in the notebook, and the processed and duplicate/name filtered files created 60 labelled training clips (15 per activity) and two distinct test sets.

2.1 Signal alignment and cleaning

- The measured raw sampling rate of the accelerometer was about 100.2 Hz. The greatest common time interval allows both sensors to be put on the same 60 Hz grid via linear interpolation.
- Single recordings of the tests were kept as long sequences. Activity labels were placed on their activity based on the 15-second segments the notebook declared.

2.2 Sampling rate and window logic

I resample the signals to 60 Hz, which produces 60 x 2 = 120 sensor samples for a 2-second feature window. Adjacent windows share half of their measurements for a 50% overlap, thus a 50% window (1-second step, 60 samples). This gives you more estimates of the sequence you could have, and still have sufficient samples to describe short movement patterns.

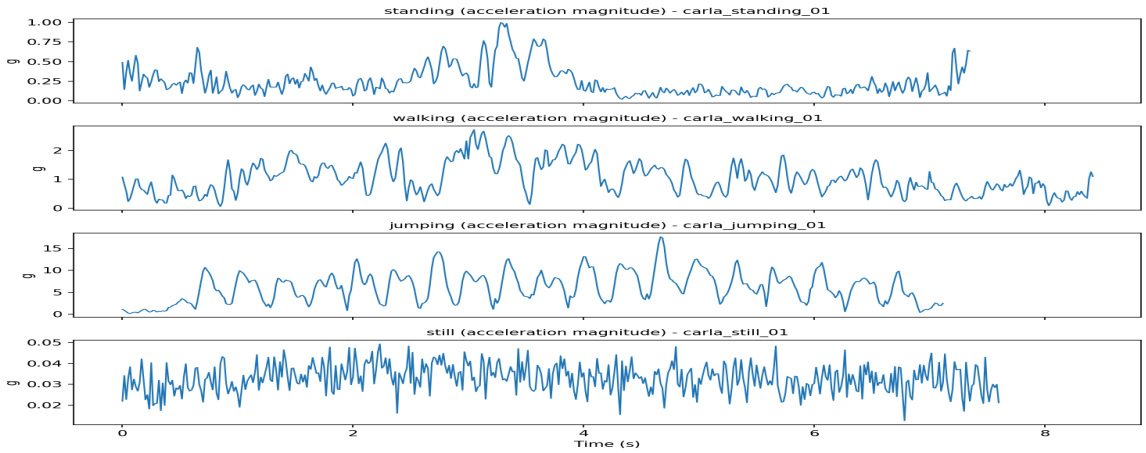


Figure 1. Example acceleration-magnitude traces for the four labelled activities, reproduced from the notebook.

2.3 Data quality

Data quality. The 60 training files last 6.1-8.6 s each (within the required 5-10 s), giving 102.8 s for standing, 105.3 s for walking, 107.2 s for jumping and 101.7 s for still, so every activity exceeds the 1 minute 30 seconds minimum. Each file is a labelled CSV with a timestamp column. All data came from one phone and one participant at a raw rate of 100.2 Hz, resampled to 60 Hz.

Evidence in the form of a video to submit:

Standing video: carla standing

Walking video: carla walking

Jumping video: Carla Jumping

Still video: carla still

3. Feature extraction and normalization

Feature	Domain	Why it may help distinguish activity
Mean magnitude	Time	Captures the typical signal level within a window.
Standard deviation	Time	Measures signal variability; active movement usually varies more than a steady phone.
Root mean square (RMS)	Time	Summarizes signal magnitude and movement intensity.
Signal magnitude area (SMA)	Time	Combines absolute axis magnitudes to represent overall movement.
X–Y axis correlation	Time/cross-axis	Captures coordinated movement between axes.
Dominant frequency (FFT)	Frequency	Identifies the strongest non-zero frequency component, potentially separating rhythmic motion.
Log spectral energy (FFT)	Frequency	Summarizes spectral power while reducing scale through a logarithm.
Spectral entropy (FFT)	Frequency	Describes how spread out spectral power is across frequencies.

The feature extraction is separately used on the acceleration and gyroscope data. The signal is mean-subtracted first and Hann-windowed before the FFT is computed. Before the calculation of dominant frequency, energy, and entropy, the zero-frequency component is removed.

3.1 Normalization

All the amplitude parameters (mean, standard deviation, RMS, and SMA) are log-transformed with scaled values. The notebook next processes the data with Z-score normalization with respect to the means and std for the training set features. This then uses Z-score normalization on the data with respect to the feature means and std of the features for the training set. The same training statistics as applied before are used during storage for the test data set; thus, the test set cannot yield its own scale. In order to contribute on a similar scale, features with various units and magnitudes are standardized.

4. HMM setup and implementation

The model is a diagonal-Gaussian Hidden Markov Model (HMM) of four states. The hidden state at each time step is one of the four activities, and the observation is the 16-dimensional standardized feature vector from the window.

HMM component	**Implement in notebook**
Hidden states	Four activity states in fixed order: standing, walking, jumping, still.
Observation	16 standardized sensor features per window.
Initial probabilities	Estimated from the first state in each labelled training sequence, with smoothing.
Transition matrix	Estimated from consecutive labelled windows, with additive smoothing, then refined by Baum–Welch.
Emission	Diagonal Gaussian distributions with a mean and variance for each state-feature pair; a variance floor avoids zero variance.
Training	Forward–backward expectation-maximization (Baum–Welch), stopping when relative log-likelihood change is below 0.0001 or max iterations are reached.
Decoding	Viterbi algorithm finds the most likely sequence of hidden activity states.

4.1 Why these algorithms fit the task

Viteribe is suitable for decoding the most probable path of the current activity using temporal transition patterns and the sensor properties in each window. Rather, options for transition and emission parameters are reduced by maximizing sequence likelihood, as done in Baum–Welch. The convergence check offers a helpful stopping rule, and the notebook documents that the training-only validation model converged after 7 iterations, and the final model after 8 iterations.

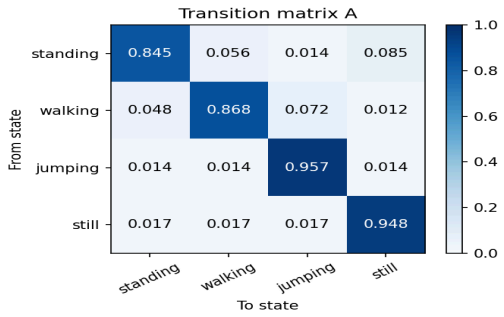


Figure 2. Learned transition probabilities; rows are the previous state and columns are the next state.

Advantageous diagonal parts in the transition matrix: 0.845 for standing, 0.868 for walking, 0.957 for jumping, and 0.948 for still. This reflects the recording design, with each recording having primarily one activity. The model thus expects a window to be most likely to stay the same from one frame to the next.

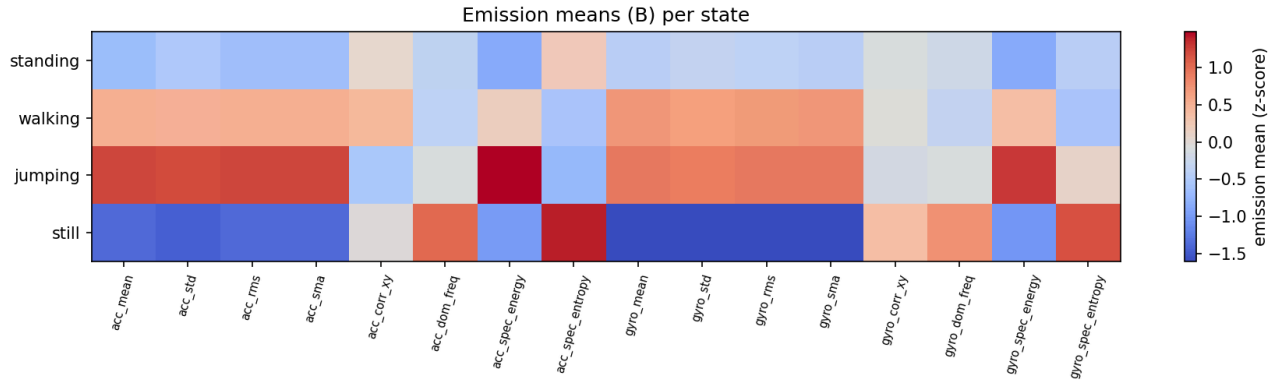


Figure 3. Emission-mean heatmap for the 16 standardized features across the four states.

5. Results and evaluation on unseen data

The notebook assesses two test recordings not used for model development. The recordings are each about 61 seconds, and have four separately highlighted activity times of 15 seconds each in an unpredictably varying sequence. Each test window can be given its reference label using the window centre.

Activity	Sample	Sensitivity	Specificity	One vs rest accuracy
Standing	30	0.800	0.955	0.915
Walking	30	0.900	0.875	0.881
Jumping	29	0.897	0.955	0.941
Still	29	0.759	1.000	0.941
Overall	118	–	–	0.839 overall accuracy

Overall accuracy was 83.9% (0.847 and 0.831 for the two recordings)

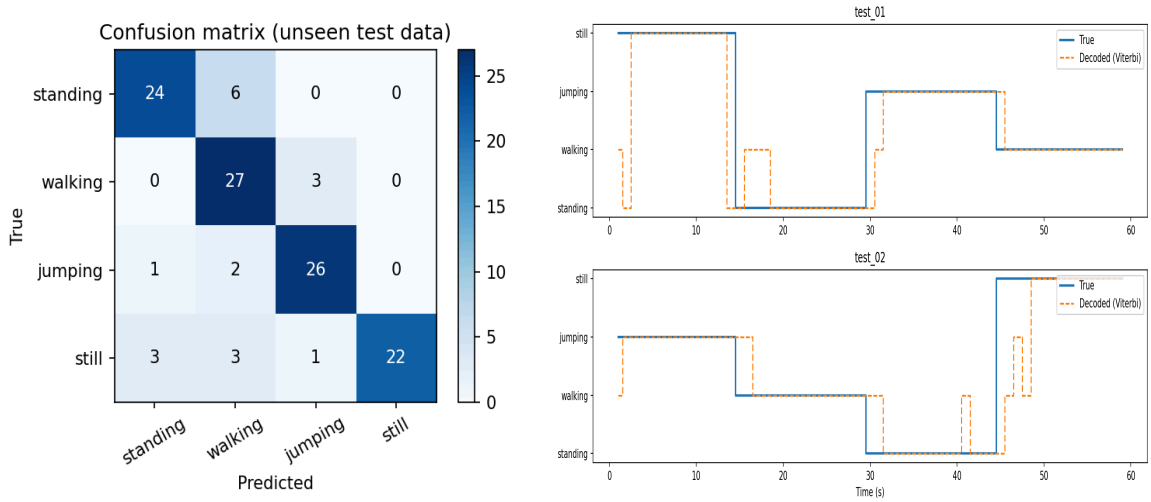


Figure 4. Confusion matrix for the two unseen test recordings.

Figure 5. True versus Viterbi-decoded activity sequences for the two unseen recordings.

6. Discussion and conclusion

Activity that is easiest and hardest. The jumping (0.897) and walking (0.900) movements were easiest to identify, as they have repeating high-energy signals that are markedly different from the flat. Still was the hardest (sensitivity 0.759): Of 29 still windows, 3 were decoded as standing, 3 as walking, and 1 as jumping, and 6 of 30 standing windows were decoded as walking. Both standing and still produce almost flat signals, and even minor hand movements are similar to the difference between standing and still. Walking has the lowest specificity (0.875) since the class tries to catch 11 windows from other classes. The specificity of Still was always 1.000, which means there were never any false predictions.

The high self-transition probabilities indicate temporal continuity and the recording system based on clips. Features may be impacted by noise, placement of phones, interpolation, and resampling from approximately 100.2 Hz to 60 Hz. The generalization of generality is unclear due to the use of only two test recordings, 15-second manual labels, and should only be generalized for other people, phones, and environments.

Conclusion. 43.7% of seen recordings were classified correctly, using a Hidden Markov Model implemented from scratch and trained with the Baum-Welch algorithm and decoded with the Viterbi algorithm from three activities based on data registered by the phone's motion sensor. Jumping and walking could be recognized with a satisfactory rate of reliability, and standing still is the primary problem.